

HOPE-CIFAR

Hierarchical Online Plasticity Engine

CIFAR-100 | Class-Incremental Continual Learning

Geliştirme & Deney İlerleme Raporu — Mayıs 2026

EN İYİ SONUÇ

56.69%

Avg Accuracy (Replay + Gaussian)

EN AZ UNUTMA

10.30%

Forgetting ($\text{lr}=1\text{e-5}$, 5 epoch)

Proje Özeti

HOPE, 100 sınıflı CIFAR-100 üzerinde Class-Incremental Learning (CIL) problemi için geliştirilmiş bir mimaridir. Model, 10 göreve bölünmüş 100 sınıflı sıralı olarak öğrenir ve her yeni görev öğrenilirken eski görevlerin unutulmaması hedeflenir.

Bu rapor; mevcut geliştirmeleri, çalıştırılan deneyleri ve laboratuvar GPU ortamında beklenen sonuçları özetlemektedir.

Mimari Genel Bakış

Görüntüden özelliğe, özellikten sınıfa akış:

```
CIFAR-100 Görüntüsü (32x32 veya 224x224)
|
[Backbone] ResNet18 (512-dim) veya ViT-B/16 (768-dim)
|
[CMS] 4 Kademe: fast(1) -> mid(4) -> slow(32) -> ultra(128)
|
[Classifier] Linear(512/768 -> 100 sınıf)
```

CMS (Continual Memory System)

- Hızlı kademe: tek epoch içinde güncellenir, yeni göreve hızla uyum sağlar
- Yavaş kademeler: uzun vadeli bilgiyi korur, görev sınırında sıfırlanmaz
- DeepMomentum optimizier: her kademe için bağımsız öğrenme dinamiği

Öğretme Sinyali (Teach Signal)

- CE kayıp gradyanının kapalı-form hesabı: backprop olmadan CMS güncellenir
- W snapshot: backward öncesi alınır, matematiksel tutarlılık sağlar

Yapılan Geliştirmeler

1. Hata Düzeltmeleri (6 Kritik Bug Fix)

- Bug 1:** CalibrationBuffer.add() eksikti — buffer hiç dolmuyordu
- Bug 2:** CMS update backward'dan önce yapılıyordu — computation graph bozuluyordu
- Bug 3:** NCM, cms_out yerine backbone_feat kullanmıyordu — CMS katkısı ölçülüyordu
- Bug 4:** LayerNorm parametreleri CMS gradyanından hariçti — norm katmanları öğrenemiyordu
- Bug 5:** Teach signal W snapshot almıyordu — doğru gradient hesaplanamıyordu
- Bug 6:** num_workers macOS uyumlu hale getirildi (0 olarak sabitlendi)

2. Gaussian Classifier Alignment (TUNA'dan Uyarılma — ICCV 2025)

TUNA makalesindeki classifier_align() yöntemi ResNet/ViT mimarisine uyarlandı. Gerçek görüntü saklamak yerine her sınıfı Gaussian dağılımla modeller.

- memory/gaussian_buffer.py — GaussianBuffer: mean + covariance her sınıf için
- training/classifier_align.py — CosFaceLoss + classifier_align() fonksiyonu
- Her görev sonunda TÜM görülen sınıfların istatistikleri yenilenir (backbone drift düzeltmesi)

[Görev T bitti]

```
-> Tüm görülen görevlerin train loader'larından backbone_feat topla
-> Her sınıf için mean + covariance hesapla
-> MultivariateNormal.sample() ile 256 sentetik örnek/sınıf üret
-> Classifier'ı SGD + CosineAnnealingLR ile 30 epoch kalibre et
```

3. CosFace Loss

Angular penalty loss — cosine benzerliğine margin ekler, sınıflar arası açıklık artar.

```
m=0.0 (varsayılan): scaled cosine CE -> s * cos(W, f)
m>0.0                : doğru sınıfın cosine değerinden m çıkarılır
Varsayılan: s=20.0, m=0.0
```

4. Çoklu Backbone Desteği (--backbone resnet/vit)

ResNet18: 32×32 CIFAR girişi, 512-dim, Mac/CPU için optimize, hızlı test

ViT-B/16: 224×224 girişi, 768-dim, ImageNet-21k ön-eğitim, lab GPU için

5. GPU Odaklı Eğitim (train_gpu.py)

- Mixed Precision AMP (fp16) — yaklaşık 2x hız artışı, daha az VRAM kullanımı
- pin_memory=True + num_workers=4 — CPU→GPU veri transfer hızlandırması
- torch.backends.cudnn.benchmark — sabit input için cuDNN otomatik kernel seçimi
- Gradient accumulation (--accum_steps) — küçük GPU'da büyük batch etkisi
- CUDA kontrolü: GPU yoksa çalışmayı reddeder, net uyarı verir

6. Dağıtık Eğitim (train_distributed.py)

- PyTorch DDP (DistributedDataParallel) ile çok-GPU desteği
- CMS parametreleri: manuel all_reduce ile senkronize (autograd dışında)
- Checkpoint kayıt/yükleme: --resume ile kesilen eğitim devam eder

```
torchrun --nproc_per_node=4 train_distributed.py --backbone vit ...
```

7. Yeni Komut Satırı Bayrakları

- backbone** resnet/vit — backbone seçimi
- gaussian_align** Gaussian kalibrasyon etkinleştir
- cosface** CosFace loss kullan
- align_epochs** Kalibrasyon epoch sayısı (varsayılan: 30)

<code>--cosface_scale</code>	CosFace ölçek faktörü (varsayılan: 20.0)
<code>--accum_steps</code>	Gradient accumulation adım sayısı (GPU için)
<code>--no_amp</code>	Mixed precision kapat (debug için)

Deney Sonuçları — ResNet18, CIFAR-100 (10 Görev × 10 Sınıf)

Konfigürasyon	Avg Acc	Forgetting	Temel Bulgu
Baseline (önceki en iyi)	44.50%	11.56%	Referans
gaussian_align (tek başına)	36.24%	36.36%	Backbone drift problemi
gaussian_align + cosface	27.90%	46.48%	CE/CosFace uyumsuzluğu
gaussian_align + freeze_backbone	22.20%	10.68%	ResNet özellikleri yetersiz
gaussian_align + lr=1e-5 + 5 epoch	44.88%	10.30%	Baseline eşiti, az unutma
gaussian_align + replay	56.69%	16.64%	EN İYİ SONUÇ (+12.2%)

En İyi Konfigürasyon: gaussian_align + replay (Avg Acc: 56.69%)

Görev 9 sonrası tam accuracy matrisi (satır = o görevin ardından, sütun = test görevi):

After / Test	T00	T01	T02	T03	T04	T05	T06	T07	T08	T09
T00	85.3									
T01	74.6	75.5								
T02	69.1	69.4	77.8							
T03	64.7	62.9	72.8	70.2						
T04	61.4	59.8	69.7	66.3	72.2					
T05	59.3	56.7	66.8	62.7	62.8	70.9				
T06	55.3	53.7	63.0	58.5	59.7	65.3	70.8			
T07	53.9	51.3	60.8	57.3	57.9	62.5	63.6	63.4		
T08	54.5	50.1	61.0	56.2	56.0	60.3	58.9	60.1	66.3	
T09	52.9	49.9	59.0	53.0	55.0	56.8	56.9	55.3	63.4	61.6

Temel Bulgular

- Replay + Gaussian alignment kombinasyonu en güçlü sonucu verdi (56.69%). Replay backbone'u eski sınıflar için kararlı tutuyor; Gaussian ise classifier kaymasını düzeltiyor.
- CosFace tek başına zarar verdi: backbone CE ile eğitilip kalibrasyon CosFace ile yapılıncaya tutarsızlık oluşuyor. ViT + frozen backbone durumunda daha uyumlu olması bekleniyor.
- Frozen backbone ResNet18 için yetersiz: ImageNet özellikleri CIFAR-100 uyarlamasız zayıf kalıyor (%22.20). ViT-B/16'da tam tersi bekleniyor — ön-eğitim özellikleri çok daha güçlü.
- Backbone drift asıl sorun: Gaussian ortalamalar her görev sonunda TÜM görülen görevler için yenilenmezse eski sınıfların prototipleri geçersiz kalıyor.

Lab GPU Beklentileri — ViT-B/16 + Dağıtık Eğitim

Referans: TUNA (ICCV 2025)

TUNA (Frozen ViT-B/16 + task-specific adapters + universal adapter) CIFAR-100'de 92.15% final accuracy rapor etmiştir. Bizim yaklaşımımız adapter mekanizması içermemekle birlikte Gaussian alignment ve CMS ile rekabetçi sonuçlar hedeflenmektedir.

Neden ViT Daha İyi?

Özellik	ResNet18	ViT-B/16
Özellik boyutu	512-dim	768-dim
Ön-eğitim verisi	ImageNet-1K	ImageNet-21K (14x büyük)
Bağlam	Konvolüsyon (yerel)	Self-Attention (global)
Frozen performans	Zayıf (~%22)	Güçlü (~%70-80 beklenti)

Beklenen Doğruluk Aralıkları

Konfigürasyon	Avg Acc	Forgetting	Not
ViT + freeze + gaussian_align	%70–78	%12–18	TUNA mantığıyla tutarlı
ViT + freeze + gaussian_align + cosface	%72–80	%10–16	CosFace frozen ViT'e uygun
ViT + replay + gaussian_align	%78–85	%10–15	En güçlü beklenti
ViT + DDP (4 GPU) + replay + gaussian	%80–87	%8–13	Tam lab konfigürasyonu

Lab Çalıştırma Komutları

Tek GPU (test ve doğrulama):

```
python3 train_gpu.py --backbone vit --gaussian_align --freeze_backbone
python3 train_gpu.py --backbone vit --gaussian_align --cosface --freeze_backbone
python3 train_gpu.py --backbone vit --gaussian_align --replay --samples_per_class 50
```

4 GPU dağıtık eğitim (tam konfigürasyon):

```
torchrun --nproc_per_node=4 train_distributed.py \
  --backbone vit --gaussian_align --cosface \
  --replay --samples_per_class 50 \
  --epochs 15 --batch_size 64
```

Küçük GPU (8GB VRAM) için — gradient accumulation:

```
python3 train_gpu.py --backbone vit --gaussian_align \
  --batch_size 32 --accum_steps 4 # efektif batch = 128
```

Hız Tahmini (4× NVIDIA A100 80GB)

ResNet18, 10 epoch/görev, 10 görev — 1 GPU

~8 dakika

ViT-B/16, 15 epoch/görev, 10 görev — 1 GPU

~45 dakika

ViT-B/16, 15 epoch/görev, 10 görev — 4 GPU

~12 dakika

+ Gaussian alignment (30 ep × 10 görev)

+5 dakika

Sonraki Adımlar

1. Lab'da ViT + freeze_backbone + gaussian_align çalıştır — ablasyon için temel al
2. CosFace margin (m=0.1, 0.2) dene — frozen ViT ile margin anlamlı olabilir
3. Gaussian n_per_class hiperparametresini karşılaştır: 128 vs 256 vs 512
4. DDP ile tekrarlanabilirlik testi (birden fazla seed)
5. TUNA'nın adapter mekanizmasını eklemeyi değerlendir (uzun vadeli hedef)

Proje Dosya Yapısı

```

hope_cifar/
  train.py          Ana eğitim scripti (Mac / CPU / MPS)
  train_gpu.py      GPU odaklı eğitim (AMP + pin_memory + cudnn)
  train_distributed.py Dağıtık eğitim (DDP, torchrun)

  model/
    backbone.py     ResNetBackbone + ViTBackbone
    cms.py           4-kademe CMS (fast/mid/slow/ultra)
    hope_model.py    Ana model: backbone + CMS + classifier

  training/
    engine.py        train_one_epoch, evaluate, evaluate_ncm
    classifier_align.py CosFaceLoss + classifier_align()

  memory/
    gaussian_buffer.py GaussianBuffer (mean + covariance per class)
    replay_buffer.py  CalibrationBuffer (görüntü tabanlı replay)

  data/
    cifar100.py      CIFAR-100 görev yükleme, TRANSFORM, VIT_TRANSFORM

  optim/
    deep_momentum.py DeepMomentum: CMS için özel optimizer

  utils/
    metrics.py       ContinualMetrics: accuracy matrisi + forgetting

```

Hızlı Başvuru: Komut Örnekleri

Mac test (varsayılan): `python3 train.py`

Mac + Gaussian: `python3 train.py --gaussian_align`

Mac + Gaussian + Replay (en iyi): `python3 train.py --gaussian_align --replay`

Mac + Frozen + Gaussian: `python3 train.py --gaussian_align --freeze_backbone`

Lab GPU (önerilen): `python3 train_gpu.py --backbone vit --gaussian_align --freeze_backbone`

Lab GPU + CosFace: `python3 train_gpu.py --backbone vit --gaussian_align --cosface`

Lab 4-GPU dağıtık: `torchrun --nproc_per_node=4 train_distributed.py --backbone vit ...`

Sonuç

HOPE-CIFAR, ResNet18 üzerinde 6 kritik hata düzeltilmesi, Gaussian Classifier Alignment (TUNA'dan uyarlama), CosFace loss, çoklu backbone desteği ve GPU/dağıtık eğitim altyapısıyla güçlü bir sürekli öğrenme sistemi haline getirilmiştir.

Mevcut en iyi ResNet18 sonucu: %56.69 avg accuracy / %16.64 forgetting (gaussian_align + replay ile baseline'i

12.2 puan geçtik).

Lab'da ViT-B/16 + frozen backbone + Gaussian alignment kombinasyonuyla %70–85 aralığında bir sonuç beklenmektedir. Bu sonuç, TUNA'nın %92.15 değerine yakın olmasa da adapter mekanizması olmaksızın rekabetçi bir performans olacaktır.